ECE 3710 Lab 3 – Fall 2023

Due Date: Week of October 2 before the beginning of your lab section

Objectives

The purpose of this lab is to gain experience interfacing the microcontroller with external peripherals using GPIO and learn how to use delay loops and verify their accuracy using a logic analyzer.

Overview

For this lab, you will be writing a ten-bit binary counter in assembly that increments at a frequency of 2 Hz (0.5 second period). The value of the counter will be displayed on an LED bar graph. Three buttons will be used to start, stop and reset the counter. The program will run on your microcontroller and it will interface with an LED display and switches using GPIO ports. Timing requirements can be verified using a logic analyzer.

Preparation

1. Come with the following:
    a. ECE 3710 Lab Kit
    b. NUCLEO-64 Board
    c. Jumper wires – Typically free in a box on a counter by the ECE store (Note that ribbon cable of various sizes is available for purchase at the ECE store – it can make connecting to the microcontroller much easier and it is reusable.)
2. Complete the Pre-Lab requirements and come prepared to pass off your prelab to the TA

Pre-Lab

1. Circuit design
    a. The LED display will be connected to your microcontroller via a breadboard; power for the LEDs will be supplied by an external voltage source.

b. You will connect one ground pin from your microcontroller and the voltage source to the breadboard (this allows the voltage source and the microcontroller to share a common ground).

c. Read over the datasheets for the ten LED bar graph and bar resistor to determine how to connect the LEDs to the microcontroller; you will need to decide which GPIO pins to use (**choose them carefully**). You will implement a 'sinking configuration' (i.e. the microcontroller is sinking current instead of supplying it). When connecting the LED display, use an active-low configuration (0 [low] means active [LED on], 1 [high] means inactive [LED off]).

d. Use three of the colored push-buttons supplied in your lab kit to control the counter. Green for start, yellow for pause, and red for reset. Carefully select GPIO pins for reading the state of the buttons. Consider how PUPDR should be configured.

e. Draw a schematic diagram of the LEDs, resistors, push-buttons, microcontroller ports, and power sources.

f. Pass the schematic diagram off with your TA before you wire it up.

g. ***Caution!*** *In general, powering external hardware through the microcontroller runs the risk of drawing too much current and then damaging the internal components. Please give careful consideration to power and current requirements of external hardware when connecting it to a microcontroller.*

Procedure

1. Software Requirements

   The counter will count at 2Hz +/- 5% as verified by the logic analyzer.

   a. When the yellow pause button is pressed, the counter will pause with the current value still visible.

   b. When the green start button is pressed, the counter will resume from the current count.

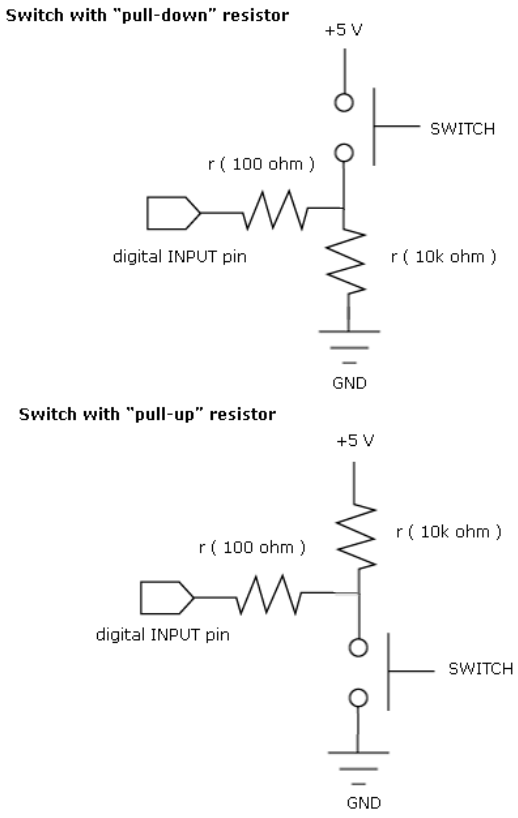
   c. When the red reset button is pressed, the counter reset to zero.

   d. Write pseudocode for the counter software. Pass off your pseudocode to the TA **before** proceeding.

2. Implementation
    a. Implement your design according to the schematic diagram and pseudocode you have developed.

3. Pass-off
    a. [5 pts] Show the functioning system to the TA
    b. [5 pts] Use the logic analyzer / oscilloscope to prove that your circuit does indeed count at a 2 Hz rate

Definitions



- Tri-state. When the GPIO pin is disabled, it enters a high-impedance state, also known as tri-state.
- Push-pull. When the pin is write-enabled without the open drain or other options enabled, it is in a push-pull configuration. The pin will source current or sink current depending on its value.
- Open drain. A configuration that works well as part of a bus because it allows multiple devices to communicate on the same line. The configuration will cause the device to sink current when active or remain in a high-impedance state when inactive.
- Pull-up vs pull-down resistor. The difference between the two configurations is the placement of the switching device; see the image. The pull-up resistor is more efficient at supplying current, while the pull-down resistor is more efficient at draining current.

Documentation

The report must include the following:

1. If applicable, a logic analyzer screen shot (if needed, review oscilloscope documentation on how to capture a screen shot) that shows the delay using the markers.

2. Pseudocode

3. The actual program code (main.s) should be included as an appendix to the report.

4. Schematic of your physical design, noting port/pin configuration.

5. Be sure that your report meets all other criteria in the Lab Grading Rubric found on Canvas.

Hints and Tips

The recommended GPIO configuration for this lab is

1. Open-drain with pull-up resistor for outputs that control LEDs

2. Input pins from the push buttons should use an internal pull-up resistor, meaning a logic '0' will be read when the button is pressed.